

🔗 本章目标

掌握 C++ 的流式 I/O:文本读写、逐行解析、字符串流、二进制读写,以及 C++17 的 `std::filesystem` 文件系统库。理解 `iostream` 相比 `stdio` 的类型安全与 RAII 优势,能够写出跨平台、健壮的文件处理代码。

📦 前置知识

熟练使用 C 的 `fopen/fprintf/fscanf` 与 `fread/fwrite`(第5章 RAII 已铺垫);了解 `string` 与 `vector`(第1、3章)。本章代码基于 C++17,文件系统库需编译器支持 `<filesystem>`。

§7.1 从 FILE 到流	§7.2 文本文件读写	§7.3 字符串流与格式化
§7.4 二进制读写	§7.5 <code>std::filesystem</code>	§7.6 错误处理与性能

§7.1 从 C 的 FILE 到 C++ 的流

C 的文件 I/O 围绕 `FILE*` 展开:`fopen` 打开, `fprintf/fscanf` 格式化读写, `fread/fwrite` 二进制读写, `fclose` 关闭。C++ 换成了流(stream)模型:`std::ofstream` 写文件、`std::ifstream` 读文件、`std::fstream` 读写兼可,配合插入运算符与提取运算符使用。流的第一个优势是类型安全:`fprintf` 的格式串与实参类型不匹配在 C 里是未定义行为,经常悄无声息地读错内存;而 `<<` 与 `>>` 是重载运算符(第6章),`int` 调 `int` 版本,`double` 调 `double` 版本,类型错了编译期就报错。

流的第二个优势是 RAII(第5章):文件流对象离开作用域自动关闭文件,不需要也不能手工 `fclose`;C 的 `fopen` 之后每条出错路径都要写一遍 `fclose`,漏一条就泄漏句柄。第三个优势是状态机:流的 `good/fail/eof/bad` 四种状态可以显式检查,把"文件读完了"与"读出错误"区分开,而 C 的 `feof/ferror` 组合很容易写错。

流的另一个实用细节是缓冲:`cout` 与文件流默认都有缓冲,写入的内容先进缓冲区,凑满或主动刷新才真正落盘。需要立即落盘(如日志、崩溃前的心跳)时调用 `out.flush()`,或者用 `endl` 换行并强制刷新——但 `endl` 有性能代价,普通换行请写 `\n`。C 的 `printf` 与 C++ 的 `cout` 混用时要注意同步:默认两者共享底层缓冲,顺序有保证,代价是性能;高频输出时可以用 `ios::sync_with_stdio(false)` 关闭同步换取速度,前提是保证不再混用 `printf`。

C 的写法	C++ 的写法	说明
<code>FILE* f = fopen(path, "r");</code>	<code>std::ifstream in(path);</code>	C++ 构造即打开,析构自动关闭
<code>fprintf(f, "%d", x);</code> 类型不符是 UB	<code>out << x;</code> 类型由重载保证	C++ 类型安全,错误编译期暴露
<code>feof/ferror</code> 组合判断	<code>in.good() / in.fail() / in.eof()</code>	C++ 流状态明确可查
<code>fclose</code> 每路径手工写	析构自动关闭	RAII 保证无句柄泄漏

清单 7-1 流的状态检查

```

#include <iostream>
#include <fstream>

int main() {
    std::ifstream in("no_such_file.txt"); // 打开不存在的文件
    if (!in) {                             // 或 in.fail():打开失败
        std::cerr << "打开失败\n";
        return 1;
    }
    std::string word;
    while (in >> word) {                    // 读到文件尾之前持续循环
        std::cout << word << " ";
    }
    if (in.eof()) std::cout << "(正常读完)\n";
    if (in.bad()) std::cout << "(读取出错)\n";
    return 0;
}

```

§7.2 文本文件读写

写文本文件用 `ofstream`:默认以截断模式打开,内容直接覆盖;需要追加时传 `std::ios::app`。打开模式可以组合,例如 `std::ios::app | std::ios::out`。打开之后必须检查流是否进入失败状态——文件路径不存在、权限不足都会导致静默失败,不检查的后果是数据凭空消失。读文本文件最常用的是 `getline`:每次取一整行,包括空格,适合逐行处理配置、CSV、日志等格式;需要按单词或数字读取时用提取运算符,它会跳过空白。

与 C 对照:`fscanf` 遇到空白就停,读"张三 85"这类行必须手写复杂的格式串;`getline` 加 `stringstream`(下节)可以把"读一行、拆字段"拆成两步,每一步都直白。另外注意 Windows 与 Linux 换行符差异:文本模式下 C++ 流自动把 `"\n"` 转换成平台换行,二进制模式不做转换,所以二进制文件必须显式加 `std::ios::binary`,否则读写内容错位。

C 的写法	C++ 的写法	说明
<code>char buf[256]; fgets(buf, 256, f);</code>	<code>std::string line; getline(in, line);</code>	C 固定缓冲易截断,C++ 动态字符串自动扩容
<code>sscanf(buf, "%s %lf", name, &v);</code>	<code>ss.str() + stringstream 拆分</code>	C 格式串脆弱,C++ 分步解析直观
<code>fopen(path, "a")</code> 追加	<code>ofstream out(path, std::ios::app)</code>	同样的追加语义,RAII 关闭

清单 7-2 写文件与逐行读取

```

#include <iostream>
#include <fstream>
#include <string>

int main() {
    // 写:截断模式;若要追加改为 std::ios::app
    {
        std::ofstream out("notes.txt");
        if (!out) return 1;
        out << "第一行:你好\n";
        out << "第二行:" << 42 << "\n";    // 数字自动转文本
    }    // 离开作用域自动关闭

    // 读:逐行
    std::ifstream in("notes.txt");
    if (!in) return 1;
    std::string line;
    int n = 0;
    while (std::getline(in, line)) {        // 读到文件尾返回 false
        std::cout << "第 " << ++n << " 行:" << line << "\n";
    }
    return 0;
}

```

⚠ 易错点 1:getline 与提取运算符混用

先 `in >> x` 再 `getline(in, line)`, `getline` 读到的是 `>>` 留下的换行,得到空行。解决:在两者之间调用 `in.ignore()` 丢弃残留字符,或者统一用 `getline` 取整行再解析,别混着用。

§7.3 字符串流与格式化

`std::stringstream` 把字符串当作流来读写:插入运算符向字符串追加内容,提取运算符从字符串取字段,`getline` 按分隔符拆列。它最常见的两个用途是拼接与解析:拼接时不需要像 `sprintf` 那样手工数缓冲大小,解析 CSV 时先把一行装进 `stringstream`,再用 `getline(ss, field, ',')` 逐个取字段。内存由 `string` 管理,不存在缓冲溢出问题。

格式化输出还有更现代的选择:`std::to_string` 把数值转字符串,适合简单拼接;C++20 的 `std::format` 提供 `printf` 风格的占位符格式化(`{}` 占位),可读性与安全性兼得;老工程里 `snprintf` 仍然常用,但记住它返回写入长度,缓冲要预留 `'\0'` 的空间。`stringstream` 的精度控制用 `out.precision(n)` 或 `<< std::setprecision(n)`(需 `<iomanip>`)。

清单 7-3 stringstream 拼接与解析

```

#include <iostream>
#include <sstream>
#include <string>
#include <iomanip>

int main() {
    // 拼接:数字与文本自由混合
    std::ostringstream report;
    report << "成绩单 " << 2026 << " 年,及格线 "
        << std::setprecision(2) << 59.5 << "\n";
    std::cout << report.str();

    // 解析:"张三,85.5" 按逗号拆字段
    std::string line = "张三,85.5,优";
    std::istringstream ss(line);
    std::string name, score, level;
    std::getline(ss, name, ',');
    std::getline(ss, score, ',');
    std::getline(ss, level, ',');
    std::cout << "姓名 " << name << ", 分数 " << score
        << ", 等级 " << level << "\n";

    return 0;
}

```

§7.4 二进制读写

二进制文件按内存字节原样存储,读写用成员函数 `write` 与 `read`:`write(const char* p, streamsize n)` 把 `n` 字节写入,`read(char* p, streamsize n)` 读 `n` 字节。典型的序列化对象是定长结构体:全部字段是数值或定长数组时,布局是确定的,直接整块读写;但有两个致命陷阱。第一,结构体有对齐填充,不同编译器、不同平台的填充不同,文件换环境就读不对——写入前用 `pragma pack` 或干脆只写字段不写结构体。第二,字节序:大端机器写的整数,小端机器读出来是反的,跨平台传输必须约定字节序并手动转换。

相比之下,`std::string`、`vector` 这类带堆指针的类型绝对不能直接 `write`:写进去的是指针值,换个进程就是悬空地址。文本格式(CSV、JSON)天然跨平台,二进制格式(图片、压缩包、协议帧)追求紧凑与速度。工程决策很简单:需要互操作选文本,追求性能且自产自销选二进制,但必须自己实现"写字段、读字段"的序列化函数,而不是裸写结构体。

两种格式的体积差异也值得记住:文本格式里数字按字符存储,一个 4 字节整数写成文本最多 11 个字符,同量级数据下二进制通常更小,而且没有解析开销;但文本格式可以直接用文本编辑器查看、用 `diff` 对比、用 `grep` 排查,可调试性远超二进制。工程上的务实做法:配置与交换格式用文本(CSV、JSON、INI),缓存与中间产物用二进制,并在二进制文件头部写版本号与魔数,将来改格式时能识别旧文件。

C 的写法	C++ 的写法	说明
<code>fwrite(p, size, n, f)</code>	<code>out.write((const char*)p, n)</code>	语义一致,字节数写法不同
<code>fread(p, size, n, f)</code>	<code>in.read((char*)p, n); in.gcount()</code>	C++ 用 <code>gcount</code> 查实际读入字节
结构体直接 <code>fwrite</code>	手写序列化函数逐字段写	C++ 更强调对齐与指针陷阱的规避

清单 7-4 二进制读写:定长记录

```

#include <iostream>
#include <fstream>
#include <cstdint>

struct Header {          // 定长布局:两个整数,无指针
    std::uint32_t magic;
    std::uint32_t count;
};

int main() {
    const char* path = "data.bin";

    // 写
    {
        std::ofstream out(path, std::ios::binary);    // 二进制必须声明
        if (!out) return 1;
        Header h{0x12345678, 3};
        out.write(reinterpret_cast<const char*>(&h), sizeof(h));
        std::uint16_t vals[3] = {10, 20, 30};
        out.write(reinterpret_cast<const char*>(vals), sizeof(vals));
    }

    // 读
    std::ifstream in(path, std::ios::binary);
    if (!in) return 1;
    Header h;
    in.read(reinterpret_cast<char*>(&h), sizeof(h));
    std::uint16_t vals[3] = {0, 0, 0};
    in.read(reinterpret_cast<char*>(vals), sizeof(vals));
    std::streamsize got = in.gcount();                // 实际读到的字节数

    std::cout << "magic " << std::hex << h.magic << std::dec
        << ", count " << h.count << ", 读到 " << got << " 字节\n";
    for (auto v : vals) std::cout << v << " ";
    std::cout << "\n";
    return 0;
}

```

⚠ 易错点 2:裸写结构体进文件

直接 `out.write((char*)&s, sizeof(s))` 的三大雷区:对齐填充因编译器而异、字节序因平台而异、string 与指针成员写进去的是地址。结果就是“本机自产自销偶尔能用,跨机器必炸”。正确的二进制序列化只写基本类型字段,且逐字段、约定字节序。

§7.5 std::filesystem:文件系统操作

C 操作目录是跨平台噩梦:stat 取信息、dirent 遍历目录、mkdir 在不同平台签名还不一样,还得自己拼接路径字符串。C++17 的 `std::filesystem` 提供了统一、跨平台、类型安全的文件系统接口:fs::path 类型管理路径(自动处理分隔符,Windows 反斜杠与 Linux 斜杠都不怕),exists、is_directory、file_size 查信息,create_directories 递归建目录,directory_iterator 遍历目录,remove、rename 删除与改名。所有函数都有异常版本与 error_code 版本:异常版本默认抛 `fs::filesystem_error`,适合正常流程;error_code 版本不抛异常,适合高频路径与“不存在不算错”的场景。

与 C 对比:opendir/readdir 需要自己判断类型、拼路径、处理隐藏文件,而 directory_iterator 直接给出完整条目对象,entry.path() 就是绝对路径,entry.is_directory() 直接判断。递归遍历用 recursive_directory_iterator,一行声明顶 C 的一页代码。路径拼接用 "/" 运算符:dir / "scores.csv",自动处理分隔符——C 里手拼路径字符串在 Windows 上经常因为漏写反斜杠出 bug。

C 的写法	C++ 的写法	说明
stat(path, &st); st.st_size	fs::file_size(path)	C 填结构体再取字段,C++ 一步到位
DIR* d = opendir(...); readdir(d)	fs::directory_iterator(dir)	C 循环读条目,C++ 范围 for 遍历
mkdir 各平台签名不同	fs::create_directories(p)	C 跨平台苦旅,C++ 统一且可递归
strcat 拼路径	fs::path dir / "a" / "b.txt"	path 类型自动处理分隔符

清单 7-5 filesystem 目录操作

```
#include <iostream>
#include <filesystem>

namespace fs = std::filesystem;      // 惯例:命名空间别名

int main() {
    fs::path dir = "backup";          // path 类型
    fs::create_directories(dir / "sub"); // 递归创建

    std::ofstream(dir / "sub" / "log.txt") << "hello\n"; // 临时文件流

    std::error_code ec;               // 不抛异常版本
    if (!fs::exists(dir, ec)) {
        std::cout << "目录不存在(错误码 " << ec.message() << ")\n";
        return 1;
    }

    for (const auto& entry : fs::recursive_directory_iterator(dir)) {
        std::cout << (entry.is_directory() ? "[目录] " : "[文件] ")
            << entry.path().string() << " "
            << fs::file_size(entry.path()) << " 字节\n";
    }

    fs::remove_all(dir);              // 递归删除,清理现场
    return 0;
}
```

 **易错点 3:目录删除与文件存在性检查的语义**

fs::remove 只能删空目录,删非空目录抛异常;需要整树删除用 fs::remove_all。另外"检查 exists 后再操作"存在竞态窗口(检查与操作之间文件可能消失),严格场景直接操作并捕获 filesystem_error,或使用 error_code 版本判断结果。

§7.6 错误处理与性能

流的错误处理要分层:打开失败检查流状态;每次写入后若担心磁盘问题,检查 `out.good()`;读取时区分 `eof`(正常读完)与 `fail`(格式错误)——提取运算符读到类型不匹配的输入会把流置为失败态,此时数据还在缓冲里,可以用 `in.clear()` 复位后继续处理。与 C 的 `errno` 体系相比,流状态是对象自带的,不会在多个调用之间互相污染。

性能方面,高频小写入尽量合并:每写一行都触发系统调用很慢,一次构造整块字符串再写,或依赖流内部缓冲;读文件按行处理比按字符快一个量级,大文件用内存映射(如 `mmap`)或流式读取,别把整个文件读进内存。I/O 是程序里最慢的部分之一,能少读一次就少读一次,能用一次读大块就不用多次读小块。

最后补一个高频陷阱的预防:程序崩溃或断电时,缓冲区的数据会丢失,文件停留在上次刷新的状态。对关键数据要养成“写入后立即刷新或关闭”的习惯,并考虑先写临时文件再原子改名,避免读到半截文件。这套“写临时文件、校验、原子替换”的手法在配置文件更新、状态持久化等场景是标准答案,配合 `filesystem` 的 `rename` 即可实现。

文件 I/O 速查表

概念	语法示例	要点
写文本	<code>std::ofstream out(path);</code>	默认截断,RAII 自动关闭
读文本	<code>std::ifstream in(path);</code>	打开失败要检查流状态
追加模式	<code>ofstream out(p, std::ios::app)</code>	内容续写在文件末尾
逐行读取	<code>while (getline(in, line))</code>	行内含空格,自动扩容
提取数据	<code>in >> x;</code> 跳过空白	类型安全,失败置 <code>fail</code>
字符串流	<code>std::stringstream ss(line)</code>	拼接与拆分字段两用
二进制读写	<code>out.write(p, n); in.read(p, n);</code>	必须加 <code>std::ios::binary</code>
读入字节数	<code>in.gcount()</code>	<code>read</code> 实际读到的数量
路径类型	<code>std::filesystem::path</code>	跨平台分隔符处理
判断存在	<code>fs::exists(p, ec)</code>	<code>error_code</code> 版本不抛异常
创建目录	<code>fs::create_directories(p)</code>	可递归创建多级目录
遍历目录	<code>fs::directory_iterator(d)</code>	<code>recursive_</code> 前缀可递归
文件大小	<code>fs::file_size(p)</code>	目录上调用会抛异常
删除文件	<code>fs::remove(p); fs::remove_all(p)</code>	<code>remove</code> 不删非空目录
流状态	<code>good / fail / eof / bad</code>	读完了 <code>eof</code> ,格式错是 <code>fail</code>

自测题

- Q 1. `iostream` 相比 `stdio` 有哪些优势?为什么说它是类型安全的?
- Q 2. `getline` 与 `>>` 混用会有什么问题?如何规避?
- Q 3. 二进制文件为什么要显式加 `std::ios::binary`?文本模式做了什么?
- Q 4. 直接 `write` 结构体到文件有什么风险?正确的二进制序列化怎么做?
- Q 5. `filesystem` 的异常版本与 `error_code` 版本如何选择?

✅ 参考答案

1. 三方面:类型安全(<< 与 >> 按重载选择版本,格式串不匹配的未定义行为消失);RAII(流析构自动关闭,无句柄泄漏);状态机(good/fail/eof/bad 显式区分正常读完与格式错误)。
2. >> 读完数据后把换行留在流里,紧接着的 getline 读到的是空行。规避:统一用 getline 取整行再解析,或中间调用 in.ignore() 丢弃残留,不要两种读法交替。
3. 文本模式在读写时做换行符转换(Windows 把 "\n" 与 "\r\n" 互转),二进制数据里的 0x0A 会被改掉,内容错位。所以二进制文件必须加 std::ios::binary 关闭转换,保证字节原样。
4. 三大风险:对齐填充因编译器而异,字节序因平台而异,string/指针成员写入的是地址。正确做法:只序列化基本类型字段,逐字段写入并约定字节序,不整块写结构体。
5. 正常流程、预期文件一定存在时用异常版本,错误信息更全;高频路径或"文件可能不存在"是正常状态时用 error_code 版本,避免异常开销与流程污染。

实验:成绩文件管理系统

本实验把本章知识串成完整的成绩管理系统:写 CSV、追加记录、逐行读取解析、统计平均分,并用 filesystem 管理数据目录与遍历。代码分三部分,覆盖点已在注释中标明,运行后 data/scores.csv 会被创建并写满记录。

实验 7•第一部分:写入(覆盖:ofstream、追加模式、打开失败检查、RAII)

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <filesystem>

namespace fs = std::filesystem;    // 覆盖:filesystem 命名空间别名

struct Record {                    // 文本记录:姓名与分数
    std::string name;
    double score;
};

// 覆盖:ofstream 写文件,append 控制覆盖或追加
bool saveCSV(const std::string& path, const std::vector<Record>& rs,
             bool append) {
    std::ofstream out(path,
                      append ? std::ios::app : std::ios::trunc);
    if (!out) return false;        // 覆盖:打开失败必须检查
    for (const auto& r : rs) {
        out << r.name << "," << r.score << "\n";    // 文本格式,跨平台
    }
    return true;                   // 覆盖:离开函数自动 close
}
```

实验 7•第二部分:读取解析(覆盖:ifstream、getline 逐行、stringstream 拆分、stod)

```

bool loadCSV(const std::string& path, std::vector<Record>& rs) {
    std::ifstream in(path);
    if (!in) return false;
    std::string line;
    while (std::getline(in, line)) {          // 覆盖:逐行读取
        if (line.empty()) continue;          // 跳过空行
        std::istringstream ss(line);        // 覆盖:stringstream 解析
        std::string name, score;
        if (std::getline(ss, name, ',') && std::getline(ss, score)) {
            try {
                rs.push_back(Record{name, std::stod(score)});
            } catch (const std::exception&) {
                std::cerr << "格式错误行:" << line << "\n";
            }
        }
    }
    return true;                             // eof 表示正常读完
}

void printAvg(const std::vector<Record>& rs) {
    double sum = 0.0;
    for (const auto& r : rs) sum += r.score;
    std::cout << "共 " << rs.size() << " 条,平均 "
                << sum / rs.size() << "\n";
}

```

实验 7·第三部分:文件系统与 main(覆盖:create_directories、exists、file_size、directory_iterator)

```

int main() {
    fs::path dir = "data";
    fs::create_directories(dir);           // 覆盖:递归创建目录
    fs::path csv = dir / "scores.csv";     // 覆盖:路径拼接

    std::vector<Record> first{"张三", 85.5}, {"李四", 59.0}, {"王五", 73.5}};
    if (!saveCSV(csv.string(), first, false)) { // 覆盖:截断写入
        std::cerr << "写入失败\n";
        return 1;
    }
    std::vector<Record> extra{"赵六", 91.0}};
    if (!saveCSV(csv.string(), extra, true)) { // 覆盖:追加写入
        std::cerr << "追加失败\n";
        return 1;
    }

    std::vector<Record> loaded;
    if (!loadCSV(csv.string(), loaded)) {
        std::cerr << "读取失败\n";
        return 1;
    }
    for (const auto& r : loaded) {
        std::cout << r.name << ": " << r.score << "\n";
    }
    printAvg(loaded);

    // 覆盖:exists 检查 + 目录遍历 + 文件大小
    if (fs::exists(csv)) {
        std::cout << "文件大小 " << fs::file_size(csv) << " 字节\n";
    }
    for (const auto& entry : fs::directory_iterator(dir)) {
        std::cout << "目录条目:" << entry.path().filename().string()
            << "\n";
    }
    return 0;
}

```

实验覆盖点核对:流类体系(ofstream/ifstream 分工)、打开失败检查、追加与截断模式、getline 逐行读取、stringstream 拆分字段、stod 与异常处理、RAII 自动关闭(三个文件流全部无手工 close)、filesystem 的 path 拼接、create_directories、exists、file_size、directory_iterator。扩展练习:把 Record 增加一列等级并修改解析;或把 saveCSV 改成二进制版本,体会两种格式的取舍。

工程建议 1:文件操作统一封装

项目里所有文件读写走少量封装函数(如本章的 saveCSV/loadCSV),统一处理打开失败、格式错误与编码问题。调用方不碰裸流,换格式(CSV 改 JSON)只改封装内部,全局调用点零修改。

工程建议 2:路径一律用 filesystem

字符串拼路径、手写分隔符判断是跨平台 bug 的高发区。路径一律用 fs::path 与 "/" 拼接,遍历用 directory_iterator,删除用 remove_all——Windows 与 Linux 的行为差异由标准库吸收,代码不再需要 #ifdef _WIN32。

本章小结

流式 I/O 取代 stdio:类型安全的运算符、RAII 的自动关闭、明确的状态机;getline 与 stringstream 把文本解析变成两步直白的操作;二进制读写记住定长与字节序;filesystem 让目录操作跨平台且类型安全。下一章进入网络通信:套接字与协议。